Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра компьютерных систем в управлении и проектировании (КСУП)

СВЯЗНЫЙ СПИСОК

Отчет по лабораторной работе №2
по дисциплине «Структуры данных»

Студент гр. 575-3
_____ И.Н. Перевозчиков
20 октября 2026 г.

Руководитель
кафедры КСУП
_____ _____ Н.А. Набережнев
оценка 22 октября 2026 г.

Томск 2026

Оглавление

Введение..... 3

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ. 4

2 МЕТОДИКА ИССЛЕДОВАНИЯ 5

Заключение 7

Введение

Цель работы – сравнительный анализ производительности двух динамических структур данных: динамического массива и двусвязного списка. В ходе работы необходимо реализовать указанные структуры, измерить время выполнения базовых операций (вставка, удаление) и на практике подтвердить теоретические оценки алгоритмической сложности.

Время выполнения операций над структурами данных зависит от двух основных факторов: размера структуры (количество элементов N) и места операции (начало, конец или середина).

В динамическом массиве элементы хранятся в непрерывном блоке памяти, что обеспечивает быстрый доступ по индексу за O(1), но вставка или удаление элемента в середине или начале требует сдвига всех последующих элементов, что занимает время O(N).

В двусвязном списке элементы (узлы) разбросаны по памяти и связаны указателями. Доступ к элементу по индексу требует последовательного перебора (O(N)), однако сами операции вставки и удаления (если известен адрес узла) выполняются за константное время O(1), без необходимости сдвигать другие элементы.

Таким образом, возникает проблема выбора оптимальной структуры в зависимости от преобладающих операций в конкретной задаче.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.

Для оценки эффективности алгоритмов используется асимптотическая сложность в нотации «О-большое» (Big-O). Она показывает, как время выполнения операции растет при увеличении количества элементов N в структуре.

Таблица 1 – Теоретическая оценка сложности операций для структур

Операция	**Динамический массив**	**Двусвязный список**
Вставка в начало	O(N)	O(1)
Вставка в конец	O(1)*	O(1)
Вставка в середину	O(N)	O(N)(поиск)/O(1)(вставка)
Удаление из начала	O(N)	O(1)
Удаление из конца	O(1)	O(1)
Удаление из середины	O(N)	O(N)(поиск)/O(1)(удаление)

2 МЕТОДИКА ИССЛЕДОВАНИЯ

Описание реализованных структур данных:

Динамический массив (структура DynamicArray): Представляет собой структуру, хранящую указатель на непрерывный блок памяти (int* _array), текущий размер (_size) и выделенную вместимость (_capacity). При добавлении элементов, если _size превышает _capacity, массив перевыделяется (вместимость увеличивается в 2 раза).

Двусвязный список (структура List): Состоит из узлов (структура Node), каждый из которых хранит данные, указатель на следующий (_nextNode) и предыдущий (_previousNode) узлы. Сама структура List хранит указатели на начало (_head), конец (_tail) и количество элементов (_size).

Обоснование выбора измеряемых операций:

Для тестирования выбраны операции вставки (AddElement / AddNode) и удаления (RemoveByIndex / RemoveNodeByIndex) из начала, конца и середины структуры.

Операции в начале и в конце демонстрируют различия в работе с граничными элементами (сдвиг всего массива против перестановки пары указателей).

Операции в середине показывают накладные расходы на поиск элемента (для списка) и сдвиг половины элементов (для массива).

Измерение времени производится с использованием библиотеки <chrono> (класс std::chrono::high_resolution_clock). Так как время выполнения одной операции может быть меньше точности таймера, используется метод усреднения: тестируемая операция выполняется в цикле M раз (например, 10 000 итераций), после чего суммарное время делится на количество итераций. Перед замерами выполняется «холостой» прогон для прогрева кэша процессора.

Замеры проводились для структур размером N=1000, 10000 и 100000 элементов.

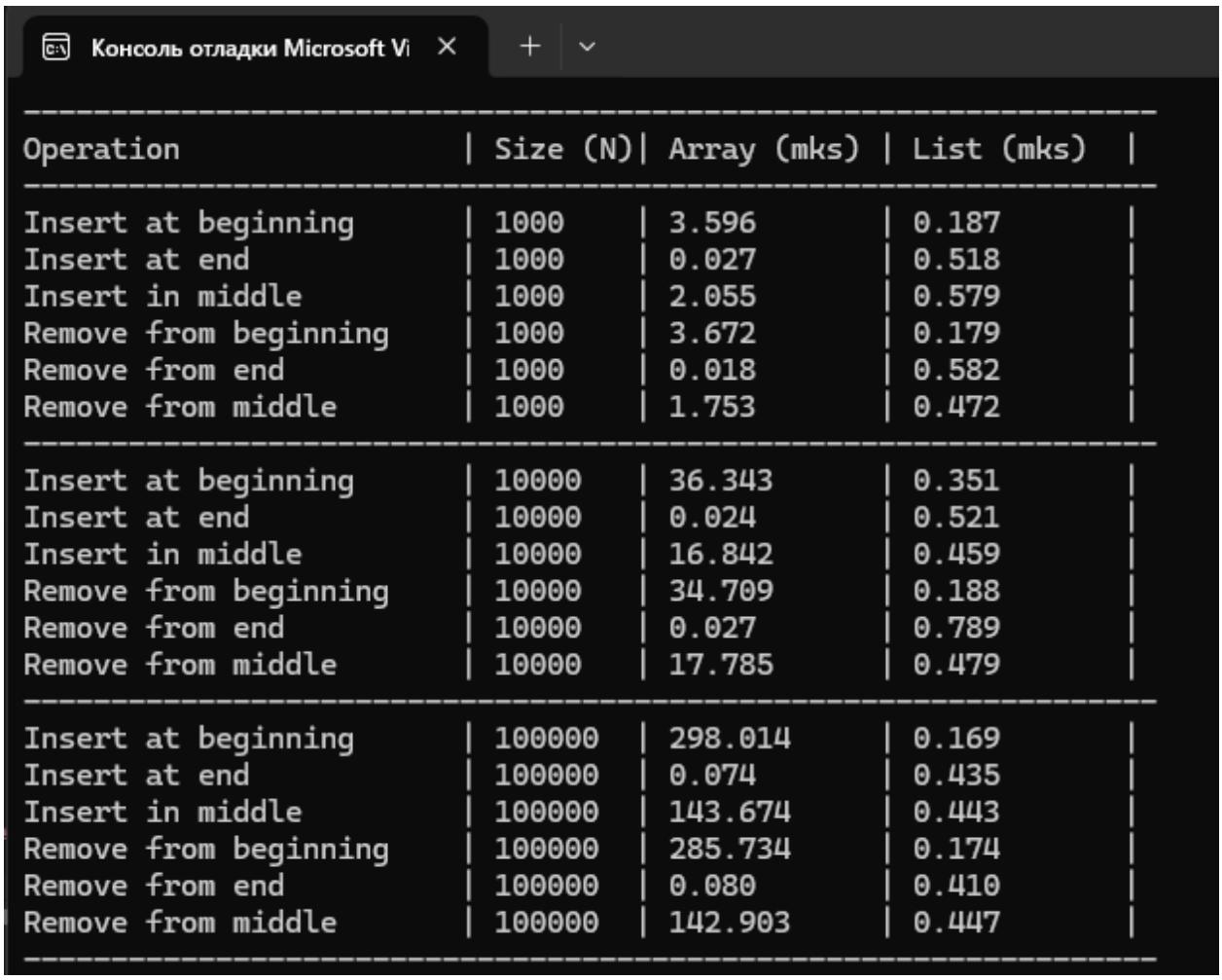


Рисунок 1 – Результаты тестирования производительности

Заключение

В ходе лабораторной работы были исследованы операции вставки и удаления для динамического массива и двусвязного списка.

В тех сценариях, где требуется частая вставка и удаление элементов в начале или в произвольных местах (при наличии итератора), эффективнее использовать двусвязный список. В сценариях с частым доступом по индексу или добавлением в конец — быстрее и экономичнее по памяти динамический массив.